

Team 6 — Distributed Dataset Statistics using MPI

Mini-Project Report

1. Problem Definition

Calculate sum, average, maximum and minimum of a large dataset using multiple MPI processes.

2. Sequential Algorithm

Generate deterministic values, traverse the dataset, accumulate sum, track maximum and minimum, calculate average, and measure elapsed time.

3. Parallel Algorithm

The root process generates the dataset. MPI_Scatterv distributes variable-sized chunks. Each process computes local sum, maximum and minimum. MPI_Reduce combines the local results at rank 0.

4. Experimental Configuration

Ubuntu master + worker1 + worker2 + worker3 VMs; MPI process counts 1, 2, 3 and 4; dataset sizes 1,000,000, 5,000,000 and 10,000,000.

5. Correctness

Average = 500.5, maximum = 1000 and minimum = 1 for every tested dataset. Expected sums are 500,500,000; 2,502,500,000; and 5,005,000,000 respectively.

6. Recorded Performance Results

Dataset	Processes	Time (s)
1,000,000	1	0.003491
1,000,000	2	0.002974
1,000,000	3	
1,000,000	4	0.012930
5,000,000	1	0.016881
5,000,000	2	0.035339
5,000,000	3	0.043528
5,000,000	4	0.051752
10,000,000	1	0.039017
10,000,000	2	0.064401
10,000,000	3	0.084561
10,000,000	4	0.095681

7. Performance Analysis

Speedup = T_1/T_p . Efficiency = $(\text{Speedup}/p) \times 100$. The recorded results show a two-process improvement for the 1M dataset, while the recorded 5M and 10M configurations become slower as process count increases. MPI

communication, synchronization, startup and virtualization overhead explain why more processes are not automatically faster.

8. Limitation

The 1M / 3-process timing was not available in the recorded experiment log, so it is intentionally left blank.

9. Conclusion

The project demonstrates distributed dataset statistics using MPI_Scatterv and MPI_Reduce and provides timing, speedup and efficiency analysis for multiple dataset sizes and MPI process counts.